

Why LLMs Become Expensive in Production

How Control-Plane Architecture Fixes It

Hiranya Abeyrathne | Technical Lead @wso2

Cloud Native Summit 2026 · Control-Plane Architecture for AI Systems



A Fraud Alert Arrives

Fraud Alert Card

**ALERT #FRD-88421**

**HIGH RISK**

 New bank account detected

 Transfers across 3 countries

 Shared device fingerprint accounts

 High-risk crypto destination

 Linked beneficiaries identified

95/100
SEVERE RISK

2024-05-19
17:35:37 UTC

AI Fraud Investigation Agent

-  **Fraud Alert Triage**
-  **Fraud Alert Investigation**
-  **Linked Account Analysis**
-  **Temporary Account Restriction**
-  **Suspicious Activity Report Draft**

A Fraud Alert Arrives

Fraud Alert Card

ALERT #FRD-88421

HIGH RISK

- New bank account detected
- Transfers across 3 countries
- Shared device fingerprint accounts
- High-risk crypto destination
- Linked beneficiaries identified

95/100 SEVERE RISK

2024-05-19 17:35:37 UTC

AI Fraud Investigation Agent



Fraud Alert Triage



Fraud Alert Investigation



Linked Account Analysis



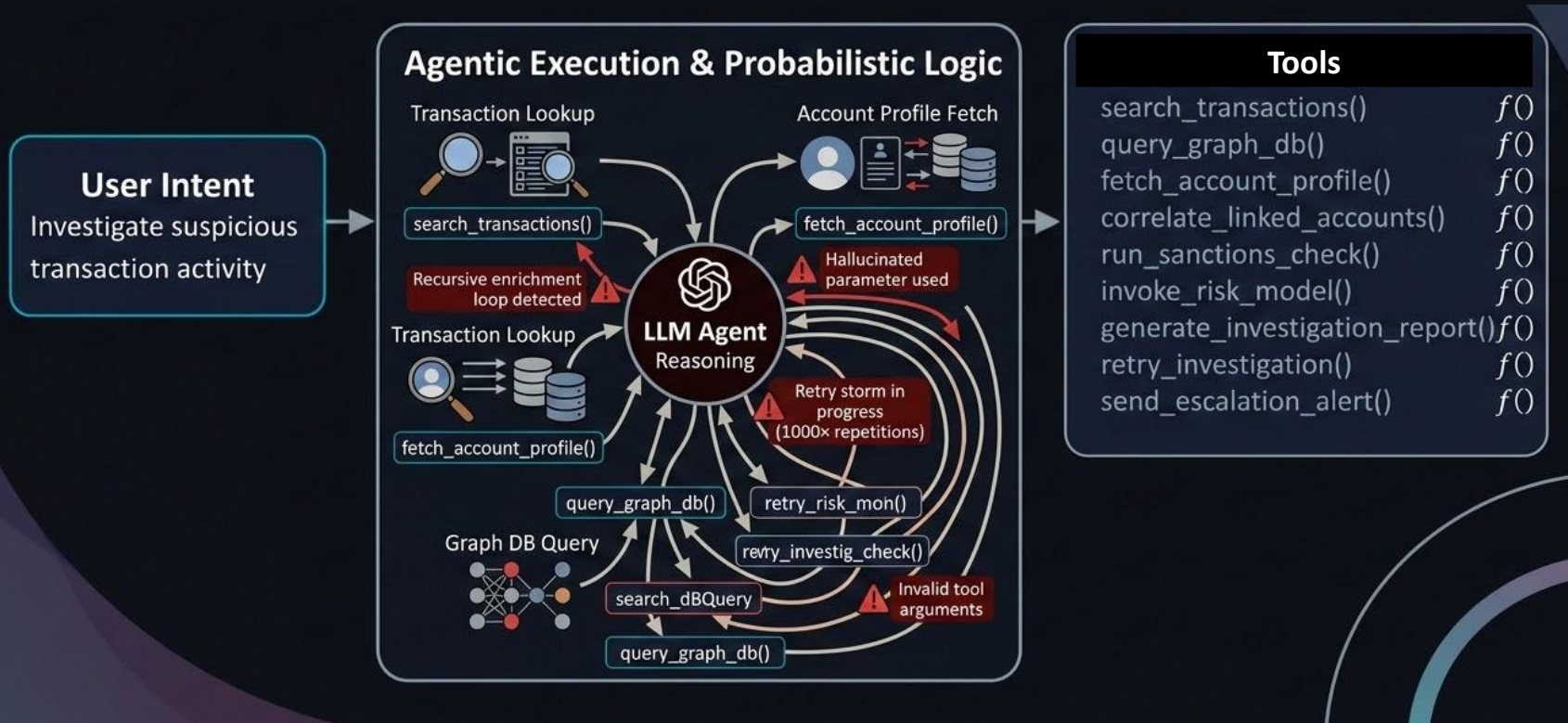
Temporary Account Restriction



Suspicious Activity Report Draft

Behind the Scenes: What the Agent Is Actually Doing

The LLM receives low-level execution primitives as tools — and must probabilistically orchestrate the workflow itself.



The Token Economics of One Fraud Alert Investigation

Token economics for an LLM-orchestrated fraud investigation workflow

· Priced at Claude Opus 4.8 standard rates (\$5 in / \$25 out per MTok). Costs vary by model.

Step	What the LLM is doing	Token usage	Estimated Cost
1	Load Fraud Case Data Reads fraud alert + tool manifests → decides to call graph DB API	~30K input + ~2K output	\$0.14
2	Correlate Linked Accounts Reads previous API responses + tool manifests → decides correlation calls → generates call parameters	~58K input + ~5K output	\$0.29
3	Sanctions & Risk Scoring Reads accumulated investigation context + tool manifests → invokes sanctions/risk scoring services → reasons over responses	~86K input + ~4K output	\$0.37
4	Generate Investigation Report Reads accumulated execution history + investigation evidence → generates investigator report	~100K input + ~14K output	\$0.57

The Token Economics of One Fraud Alert Investigation

Token economics for an LLM-orchestrated fraud investigation workflow

· Priced at Claude Opus 4.8 standard rates (\$5 in / \$25 out per MTok). Costs vary by model.

Step	What the LLM is doing	Token usage	Estimated Cost
1	Load Fraud Case Data Reads fraud alert + tool manifests → decides to call graph DB API	~30K input + ~2K output	\$0.14
2	Correlate Linked Accounts Reads previous API responses + tool manifests → decides correlation calls → generates call parameters	~58K input + ~5K output	\$0.29
3	Sanctions & Risk Scoring Reads accumulated investigation context + tool manifests → invokes sanctions/risk scoring services → reasons over responses	~86K input + ~4K output	\$0.37
4	Generate Investigation Report Reads accumulated execution history + investigation evidence → generates investigator report	~100K input + ~14K output	\$0.57

TOTAL · ~274K input · ~25K output

\$2.00 / investigation

The \$2.00M Weekend

What happens when an agentic retry loop goes undetected



Root cause: A recursive orchestration loop repeatedly re-executed the same investigation flow. No loop detection. No spend cap. No kill switch.

Autonomous orchestration

amplifies runaway infrastructure costs at a speed no human operator can catch manually.

What Actually Drives AI Execution Cost

Not the model. Not the prompt. But the execution architecture

01

Context Accumulation

Every workflow step expands future token cost

02

Autonomous Retries

Recursive retries amplify inference and API spend

03

Workflow Depth

More orchestration steps mean more reasoning cycles

04

Tool Fan-Out

Agents calling multiple tools multiply execution cost

05

Model Selection

Wrong model routing inflates runtime cost

What Actually Drives AI Execution Cost

Not the model. Not the prompt. But the execution architecture

01

Context Accumulation

Every workflow step expands future token cost

02

Autonomous Retries

Recursive retries amplify inference and API spend

03

Workflow Depth

More orchestration steps mean more reasoning cycles

04

Tool Fan-Out

Agents calling multiple tools multiply execution cost

05

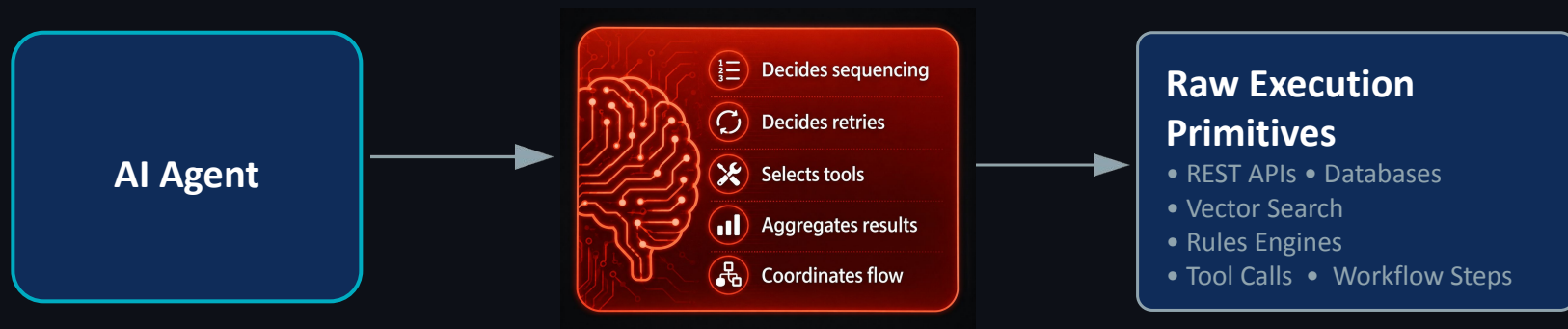
Model Selection

Wrong model routing inflates runtime cost

None of these are model problems. All five are architecture decisions.

The Core Problem: LLM as the Execution Layer

Non-deterministic reasoning + autonomous tool selection + direct access to raw primitives create governance risk

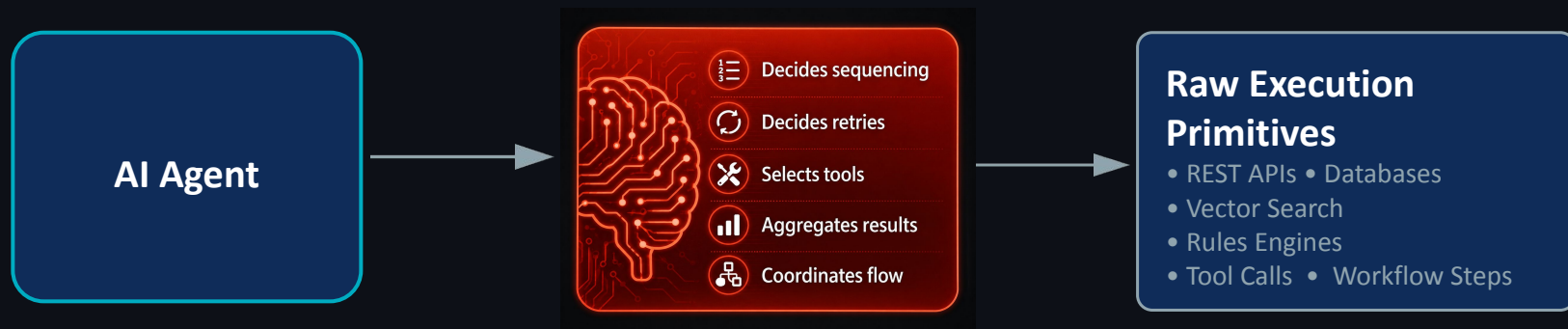


CONSEQUENCES

- ✗ Loss of deterministic control flow
- ✗ Hidden execution paths
- ✗ Recursive execution loops
- ✗ No centralized governance
- ✗ Token explosion via context accumulation

The Core Problem: LLM as the Execution Layer

Non-deterministic reasoning + autonomous tool selection + direct access to raw primitives create governance risk



CONSEQUENCES

- ✗ Loss of deterministic control flow
- ✗ Recursive execution loops
- ✗ Token explosion via context accumulation
- ✗ Hidden execution paths
- ✗ No centralized governance

Ungoverned Probabilistic Orchestration

The Impact: Operational Failure Modes

Operational failures emerge when execution decisions are delegated to the model



Identity Gap

Who is the agent? Which session? Which user? Raw MCP exposes no identity boundary.



Retry Storms

Agents retry on any failure. No human in the loop to notice a 1000× repetition.



Invalid Tool Arguments

LLMs invent tool arguments on the fly. Downstream systems receive corrupted inputs.



No Spend Ceiling

Without session-level budget enforcement, a single agent can exhaust a monthly budget in hours.

The Fix: Control-Plane Architecture for Agentic Execution

Separation of concerns for agentic systems. Reasoning, governance, and execution become three independent layers

Without control-plane architecture

model boundary

LLM

- ✗ Reasoning + orchestration tightly coupled
- ✗ Tool selection + retries
- ✗ Data aggregation + routing decisions
- ✗ Cost + policy decisions hidden in execution

Token explosion · retry storms · runaway cost

With control-plane architecture

model boundary

LLM - Reasoning only

outside the model boundary

- ✓ **Control Plane** - Runtime governance layer
 - Identity · Policy · Spend tracking · Observability
- ✓ **Platform** - Capabilities + deterministic execution
 - Sequencing · Retries · Orchestration · Compensation · Downstream calls

Bounded execution · governed spend · full observability

The Fix: Control-Plane Architecture for Agentic Execution

Separation of concerns for agentic systems. Reasoning, governance, and execution become three independent layers

Without control-plane architecture

model boundary

LLM

- ✗ Reasoning + orchestration tightly coupled
- ✗ Tool selection + retries
- ✗ Data aggregation + routing decisions
- ✗ Cost + policy decisions hidden in execution

Token explosion · retry storms · runaway cost

With control-plane architecture

model boundary

LLM - Reasoning only

outside the model boundary

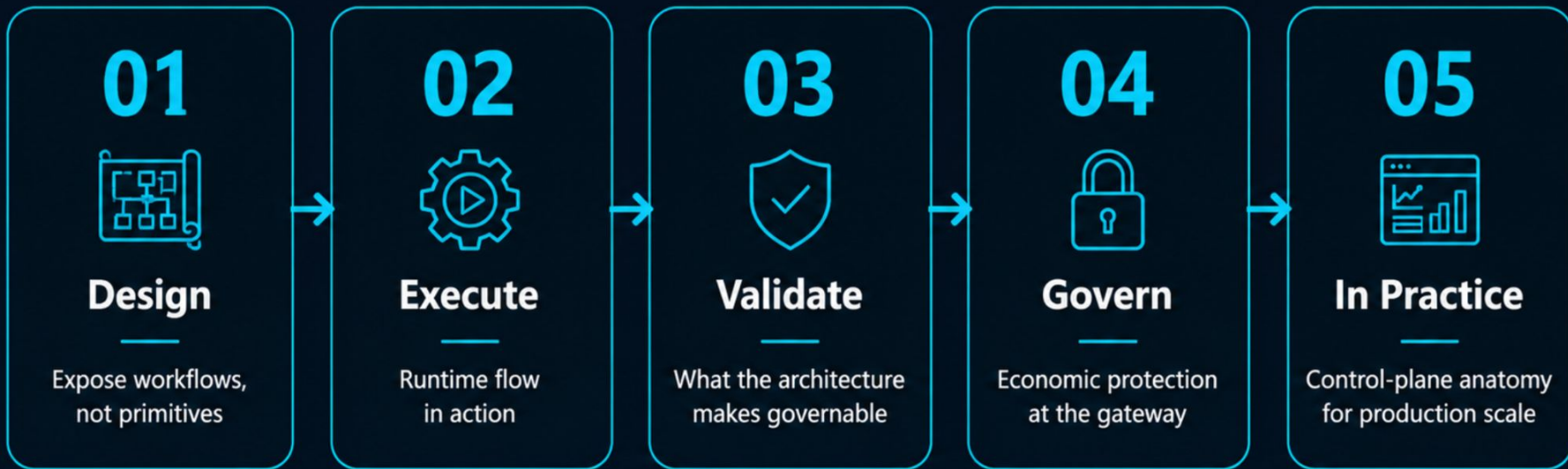
- ✓ **Control Plane** - Runtime governance layer
 - Identity · Policy · Spend tracking · Observability
- ✓ **Platform** - Capabilities + deterministic execution
 - Sequencing · Retries · Orchestration · Compensation · Downstream calls

Bounded execution · governed spend · full observability

Three concerns. Three layers. Clear separation
The LLM reasons, The control plane governs, The platform executes

The Solution Path: From Agent to Governed Execution

Five steps from architectural principle to production implementation.



By Step 5, the five cost drivers become bounded, observable, and governable

Step 1. Design: Expose Workflows, Not Primitives

Example: Expose a business workflow as a single MCP tool (`investigate_fraud_case`), with deterministic execution defined in Arazzo.

✗ Before — LLM orchestrates raw primitives

```
@tool search_transactions()
@tool correlate_accounts()
@tool run_sanctions_check()
@tool invoke_risk_model()
@tool generate_summary()
```

✓ After — LLM sees a single governed workflow tool

① Arazzo Workflow Spec

workflow: `investigate_fraud_case`
Fetch data → Risk checks → Generate summary
Retry: 3x · Timeout-aware · Protocol-agnostic

② MCP Server

Wraps workflow as an MCP tool

③ AI Gateway

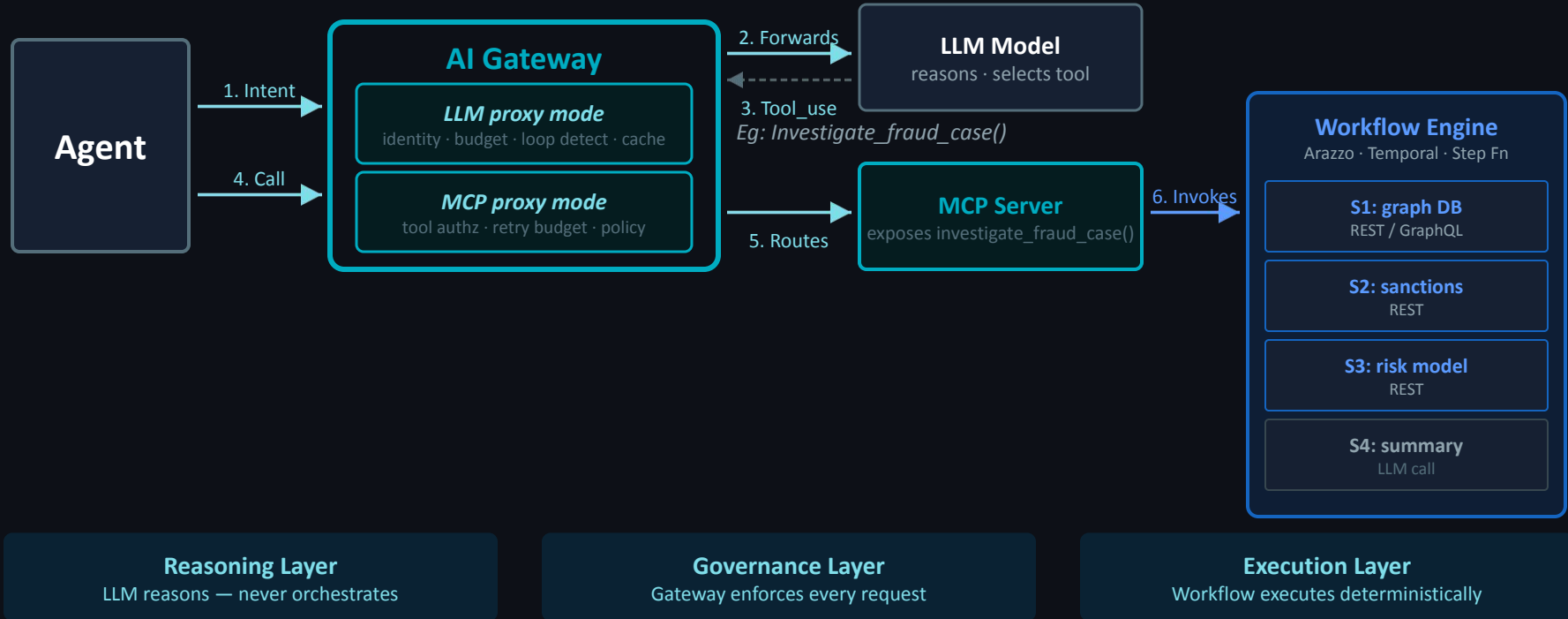
Registers and governs workflow tools

```
@tool investigate_fraud_case(case_id)
```

Build the workflows. Register them with the gateway. The LLM sees business capabilities, not raw primitives

Step 2. Execute: Agent Calls a Business Workflow

The agent reasons the intent and selects governed business capability; the workflow engine executes the steps deterministically



One tool call. Gateway-mediated execution. Deterministic workflow steps. Zero LLM-led orchestration

Step 3. Validate: Architecture Makes the Risks Governable

When orchestration moves outside the model boundary, these risks become visible, observable and governable at the platform level

Token explosion

LLM orchestrating 5 raw primitives. Tool manifests + full context on every call

Deterministic workflows + semantic caching

Hidden execution paths

No identity, no audit trail. Who called what and when?

Single workflow entry point with identity propagation and end-to-end tracing

Retry storms

LLM retries unbounded. Same tool called 1,000×

Retries become bounded via retry policies

Recursive loops

Agent re-executes same investigation flow continuously

Repeated execution patterns are now visible outside the model

Step 4A. Govern: What Runtime Boundaries We Need

Session-level enforcement at the AI gateway. Runtime controls for cost, retries, loops, and execution governance.



Token Counting & Cost Estimation

- Real-time token accumulation per session
- Pre-flight cost estimation before execution
- MCP tool usage analytics



Session Spend Tracking

- Session budget enforcement
- Execution velocity anomaly detection
- Automatic budget cutoff



Loop Detection & Kill Switch

- Detect identical request repetition
- Budget enforcement with hard cutoff
- Automated session termination

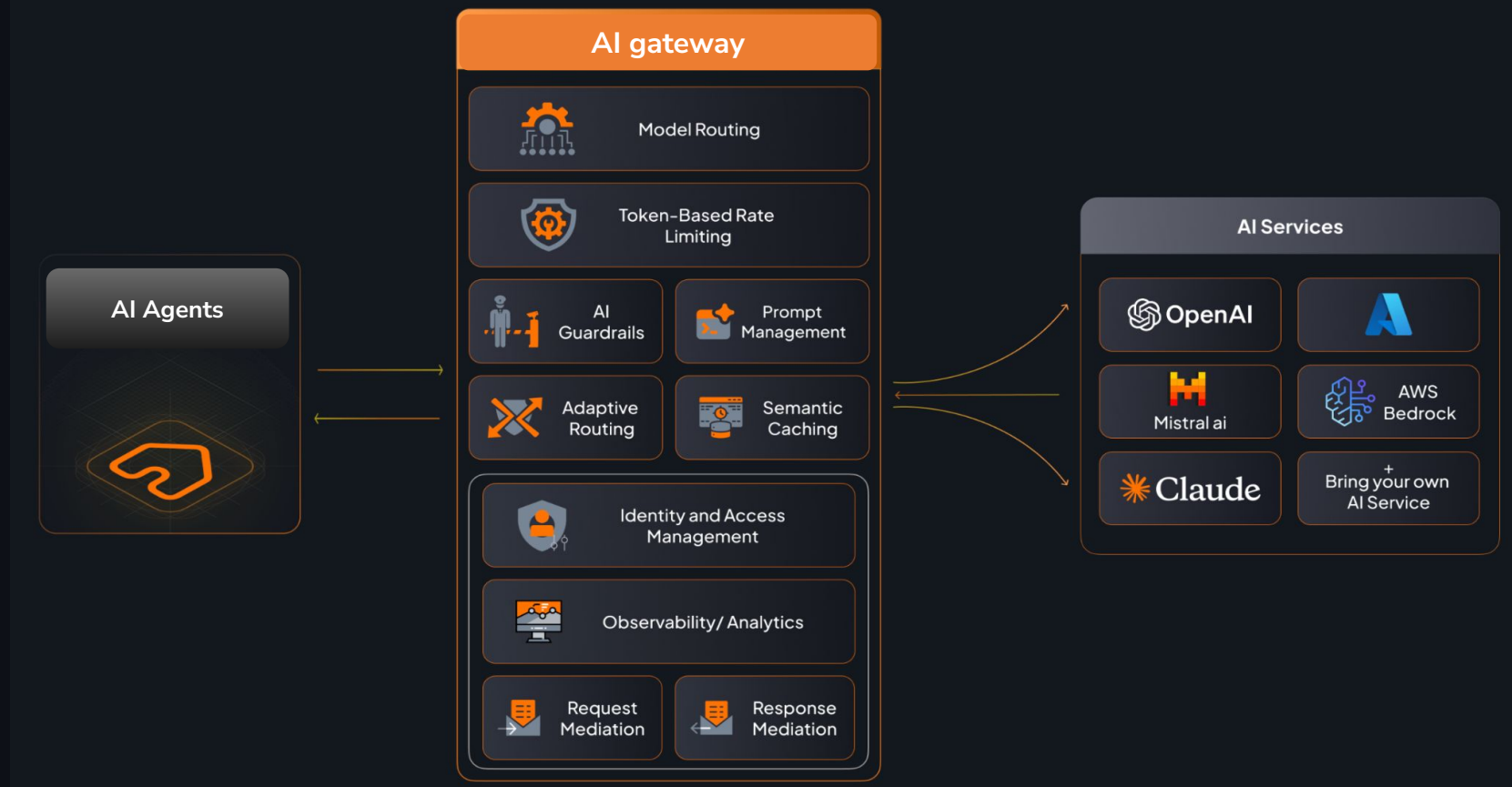


Semantic Workflow Caching & Deduplication

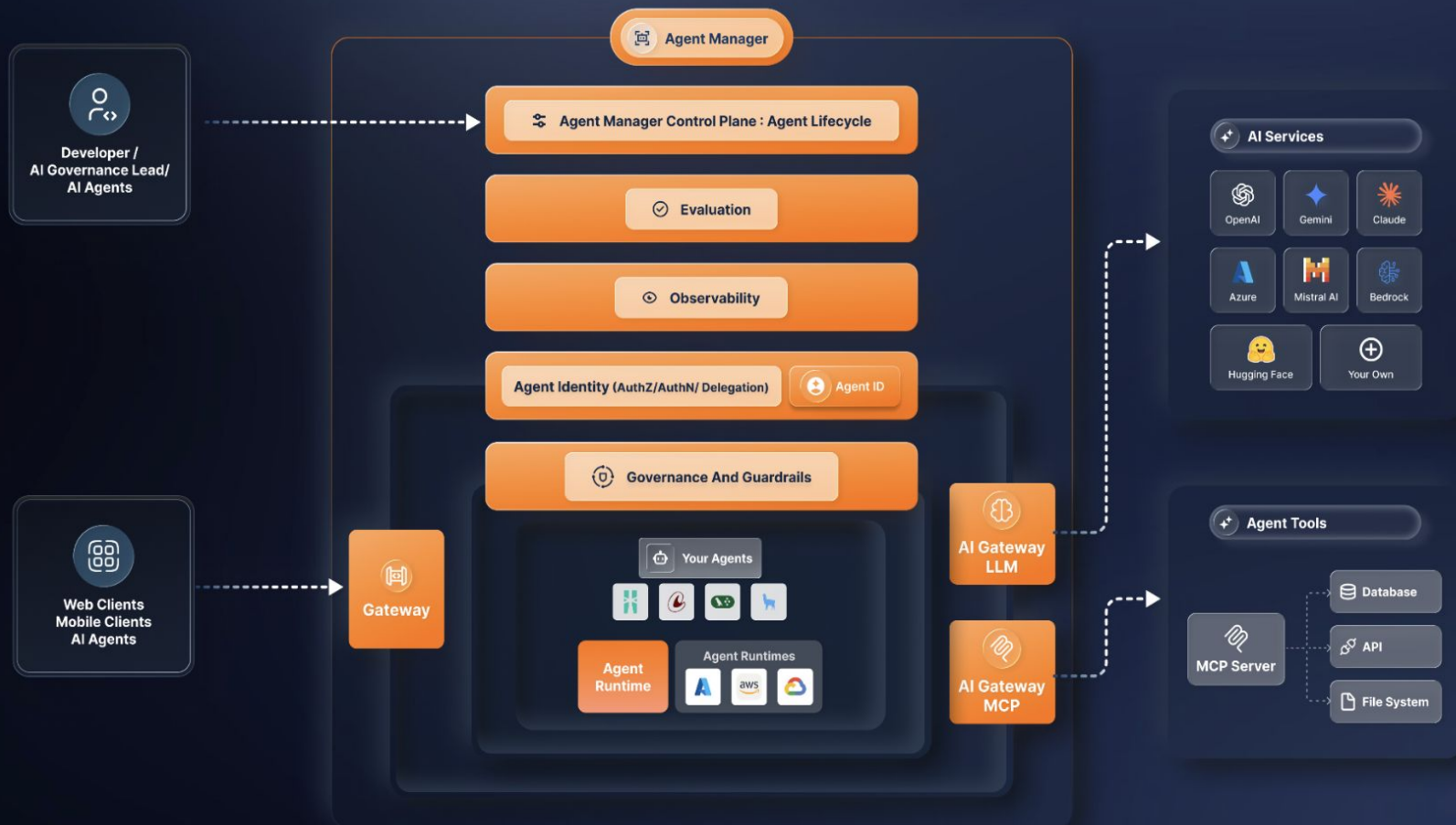
- Cache semantically identical requests
- Deduplicate concurrent tool executions
- Eliminate redundant execution across sessions

Step 4B. Govern: Runtime Governance in Practice

Runtime governance in practice. The AI gateway enforces policies, identity, observability, and execution controls.



Step 5. In Practice: Control-Plane Anatomy for Production Scale



Control-Plane Capabilities for Production Agent Systems

What an agent control plane must provide to run agent safely, predictably and economically in production

01



Agent Lifecycle & Version Control

Version, test, promote, rollback

02



Agent Identity & Access Control

Who the agent is and what it can access

03



Runtime Governance & Policy Enforcement

Budgets, approvals, limits, kill switches

04



LLM & Tool Traffic Control

Routing, caching, retries, gateway enforcement

05



Observability, Audit & Evaluation

Trace actions, cost, latency, outcomes

06



Interoperability & Standards Alignment

Multi-model, multi-framework, open interfaces

Key Takeaways

01

LLM cost and latency challenges are primarily **architectural**; not model limitations.

02

Treating LLMs as orchestration engines causes **token waste**, retry storms, and poor observability.

03

Separate reasoning from orchestration. Keep the LLM focused on intent and reasoning, while **deterministic execution layers run workflows exposed as governed tools**.

04

Control-plane architecture makes agentic execution bounded, observable, and governable by centralizing identity, policy, routing, spend control, and runtime enforcement.

You don't have a model problem. You have an architecture problem



The Model

Reasons



The Control Plane

Governs



The Platform

Executes

Questions?



Scan to access resources and contact information